LAPORAN PRAKTIKUM PEKAN 3

DEBUGGING PROGRAM GAME TEBAK ANGKA PEMROGRAMAN BAHASA C DAN PYTHON

Disusun untuk memenuhi Tugas Mata Kuliah Pemrograman Dasar

Disusun oleh:

Gunawan Cahya Ramadhan

2025806003

PRODI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS INSAN PEMBANGUNAN INDONESIA

2026

DAFTAR ISI

DAFTAR ISI 2
DAFTAR GAMBAR 3
1. PENDAHULUAN 4
2. PEMBAHASAN 5
2.1. Modifikasi Game 5
2.2. Analisis Bug pada Program Bahasa C 11
2.3. Perbaikan Bug pada Program Bahasa C 12
2.4. Analisis Bug pada Program Bahasa Python 14
2.5. Perbaikan Bug pada Program Bahasa Python 16
2.6. Teori Perbedaan For pada Program Bahasa C dan Python 17
3. KESIMPULAN 20

DAFTAR GAMBAR

Gambar 2. 1 Modifikasi Level Game pada Program Bahasa C 5
Gambar 2. 2 Modifikasi Level Game pada Program Bahasa Python 6
Gambar 2. 3 Nilai Minimal dan Maksimal Level Game pada Program Bahasa C 6
Gambar 2. 4 Nilai Minimal dan Maksimal Level Game pada Program Bahasa Python 6
Gambar 2. 5 Program Menghasilkan Angka Random pada Program Bahasa C 6
Gambar 2. 6 Program Menghasilkan Angka Random pada Program Bahasa Python 7
Gambar 2. 7 Game Tebakan Berulang pada Program Bahasa C 7
Gambar 2. 8 Game Tebakan Berulang pada Program Bahasa Python 7
Gambar 2. 9 Program Fitur Waktu Game pada Program Bahasa C 8
Gambar 2. 10 Program Fitur Waktu Game pada Program Bahasa Python 8
Gambar 2. 11 Fitur Penyimpanan Best Score pada Program Bahasa C 9
Gambar 2. 12 Program Best Score pada Program Bahasa C 9
Gambar 2. 13 Fitur Penyimpanan Best Score pada Program Bahasa Python 10
Gambar 2. 14 Program Penyimpanan Best Score pada Program Bahasa C 10
Gambar 2. 15 Proses Debugging Rumus Random pada Program Bahasa C 11
Gambar 2. 16 Hasil Pengujian Debugging pada Program Bahasa C 12
Gambar 2. 17 Perbaikan Bug (Fix) pada Program Bahasa C 12
Gambar 2. 18 Hasil Pengujian Perbaikan Bug pada Program Bahasa C 13
Gambar 2. 19 Proses Debugging Rumus Random pada Program Bahasa C 14
Gambar 2. 20 Hasil Pengujian Debugging pada Program Bahasa Python 15
Gambar 2. 21 Perbaikan Bug (Fix) pada Program Bahasa Python 16
Gambar 2. 22 Hasil Pengujian Perbaikan Bug pada Program Python 16

1. PENDAHULUAN

Pada praktikum pekan 3 ini, dilakukan pembuatan program game tebak angka menggunakan bahasa pemrograman C dan Python. Program ini dirancang untuk melatih kemampuan dalam memahami logika pemrograman, penggunaan percabangan, perulangan, serta fungsi random dalam menentukan angka yang harus ditebak oleh pengguna. Selain itu, program juga dilengkapi dengan fitur level permainan (easy, medium, hard) serta penyimpanan skor terbaik (best score) ke dalam file.

Tidak hanya fokus pada pembuatan program, praktikum ini juga mencakup proses debugging, yaitu kegiatan untuk menemukan dan memperbaiki kesalahan (bug) dalam program. Melalui proses ini, mahasiswa diharapkan mampu memahami kesalahan yang terjadi serta mengetahui cara memperbaikinya dengan menggunakan tools seperti debugger pada VS Code.

Dengan adanya praktikum ini, diharapkan mahasiswa dapat meningkatkan pemahaman mengenai konsep dasar pemrograman, pengelolaan input-output, serta pentingnya proses debugging dalam pengembangan program.

2. PEMBAHASAN

Pada praktikum pekan 3 ini, dibuat sebuah program game tebak angka menggunakan bahasa pemrograman C dan Python. Program ini dirancang untuk melatih kemampuan dalam memahami logika program, penggunaan percabangan, perulangan, serta pemanfaatan fungsi random.

Program dimulai dengan meminta pengguna memilih level permainan, yaitu easy (1–10), medium (1–100), dan hard (1–1000). Setelah level dipilih, program akan menghasilkan angka secara acak sesuai dengan rentang yang ditentukan. Pada bahasa C, digunakan fungsi rand(), sedangkan pada Python digunakan random.randint().

Selanjutnya, pengguna diminta untuk memasukkan tebakan angka secara berulang hingga menemukan jawaban yang benar. Program akan memberikan petunjuk berupa "terlalu kecil" jika angka yang ditebak lebih kecil dari angka sebenarnya, dan "terlalu besar" jika lebih besar. Setiap percobaan akan dihitung untuk mengetahui berapa kali pengguna melakukan tebakan.

Selain itu, program juga dilengkapi dengan fitur pengukuran waktu. Pada bahasa C digunakan fungsi time(), sedangkan pada Python digunakan modul time. Dengan fitur ini, pengguna dapat mengetahui berapa lama waktu yang dibutuhkan untuk menyelesaikan permainan.

2.1. Modifikasi Game

Pada praktikum ini dilakukan pengembangan program game tebak angka dengan menambahkan beberapa fitur, yaitu sistem level permainan dan penyimpanan skor terbaik (best score). Program diimplementasikan menggunakan bahasa C dan Python.

1. Penambahan Level Permainan

Program dimodifikasi agar memiliki tiga tingkat kesulitan, yaitu:

```c
printf("Pilih level:\n");
printf("1. Easy (1-10)\n");
printf("2. Medium (1-100)\n");
printf("3. Hard (1-1000)\n");
printf("Masukkan pilihan: ");
scanf("%d", &level);

if (level == 1) { min = 1; max = 10; }
else if (level == 2) { min = 1; max = 100; }
else if (level == 3) { min = 1; max = 1000; }
else {
    printf("Level tidak valid, default medium\n");
    min = 1; max = 100;
}
```

Gambar 2. 1 Modifikasi Level Game pada Program Bahasa C

```python
print("Pilih level:")
print("1. Easy (1-10)")
print("2. Medium (1-100)")
print("3. Hard (1-1000)")
```

Gambar 2. 2 Modifikasi Level Game pada Program Bahasa Python

Pengguna diminta untuk memilih level sebelum permainan dimulai. Berdasarkan pilihan tersebut, program akan menentukan nilai minimum (min) dan maksimum (max) yang digunakan dalam permainan. Sebagai contoh, pada bahasa C implementasi ini dilakukan dengan percabangan kondisi:

```c
if (level == 1) { min = 1; max = 10; }
else if (level == 2) { min = 1; max = 100; }
else if (level == 3) { min = 1; max = 1000; }
else {
    printf("Level tidak valid, default medium\n");
    min = 1; max = 100;
}
```

Gambar 2. 3 Nilai Minimal dan Maksimal Level Game pada Program Bahasa C

Sedangkan pada Python, konsep yang sama diterapkan menggunakan struktur if-elif. Dengan adanya level, permainan menjadi lebih variatif dan menantang karena pengguna dapat memilih tingkat kesulitan sesuai kemampuan.

```python
if level == 1:
    min_v, max_v = 1, 10
elif level == 2:
    min_v, max_v = 1, 100
elif level == 3:
    min_v, max_v = 1, 1000
else:
    print("Level tidak valid, default medium")
    min_v, max_v = 1, 100
```

Gambar 2. 4 Nilai Minimal dan Maksimal Level Game pada Program Bahasa Python

2. Pembangkitan Bilangan Acak

Setelah level ditentukan, program menghasilkan angka acak dalam rentang tertentu. Pada bahasa C digunakan rumus:

```c
secret = rand() % (max - min + 1) + min;
```

Gambar 2. 5 Program Menghasilkan Angka Random pada Program Bahasa C

Sedangkan pada Python digunakan:

```
secret = random.randint(min_v, max_v)
```

Gambar 2. 6 Program Menghasilkan Angka Random pada Program Bahasa Python

Kedua rumus tersebut memastikan bahwa angka yang dihasilkan selalu berada dalam rentang yang sesuai dengan level yang dipilih.

3. Proses Permainan

Program akan meminta pengguna memasukkan tebakan secara berulang hingga angka yang ditebak benar. Setiap input akan dibandingkan dengan nilai secret, kemudian program memberikan petunjuk:

```
    attempts++;

    if (guess < secret) {
        printf("Terlalu kecil\n");
    } else if (guess > secret) {
        printf("Terlalu besar\n");
    } else {
        printf("Benar! %d percobaan\n", attempts);
        break;
    }
}
```

Gambar 2. 7 Game Tebakan Berulang pada Program Bahasa C

```
        attempts += 1

        if guess < secret:
            print("Terlalu kecil.")
        elif guess > secret:
            print("Terlalu besar.")
        else:
            print(f"Benar! {attempts} percobaan.")
            break

    end = time.time()
    print("Waktu:", round(end - start, 2), "detik")

    return attempts
```

Gambar 2. 8 Game Tebakan Berulang pada Program Bahasa Python

Gambar tersebut menunjukkan proses utama permainan tebak angka pada program bahasa C dan Python. Pada kedua program, pengguna diminta untuk memasukkan tebakan secara berulang hingga angka yang ditebak sesuai dengan nilai secret.

Setiap input yang diberikan akan dibandingkan dengan nilai secret menggunakan struktur percabangan (if-else). Jika nilai tebakan lebih kecil dari secret, program akan menampilkan pesan "terlalu kecil". Sebaliknya, jika nilai tebakan lebih besar, program akan menampilkan pesan "terlalu besar".

Proses ini berlangsung secara berulang (looping) hingga pengguna berhasil menebak angka yang benar. Ketika tebakan sudah sesuai, program akan menampilkan jumlah percobaan yang telah dilakukan oleh pengguna.

Dengan demikian, gambar tersebut menggambarkan alur logika dasar permainan, yaitu perulangan input, proses perbandingan, serta pemberian umpan balik kepada pengguna hingga kondisi benar terpenuhi.

4. Pengukuran Waktu

Program dilengkapi dengan fitur pengukuran waktu permainan. Pada bahasa C digunakan fungsi time(), sedangkan pada Python menggunakan modul time. Fitur ini digunakan untuk mengetahui berapa lama pengguna menyelesaikan permainan, berikut contoh gambar:

```c
srand((unsigned) time(NULL));

secret = rand() % (max - min + 1) + min;

printf("\nTebak angka antara %d sampai %d!\n", min, max);

time_t start = time(NULL);
```

Gambar 2. 9 Program Fitur Waktu Game pada Program Bahasa C

```python
start = time.time()

while True:
    try:
        guess = int(input("Masukkan tebakan: "))
    except ValueError:
        print("Input bukan angka, coba lagi.")
        continue

    attempts += 1

    if guess < secret:
        print("Terlalu kecil.")
    elif guess > secret:
        print("Terlalu besar.")
    else:
        print(f"Benar! {attempts} percobaan.")
        break

end = time.time()
print("Waktu:", round(end - start, 2), "detik")
```

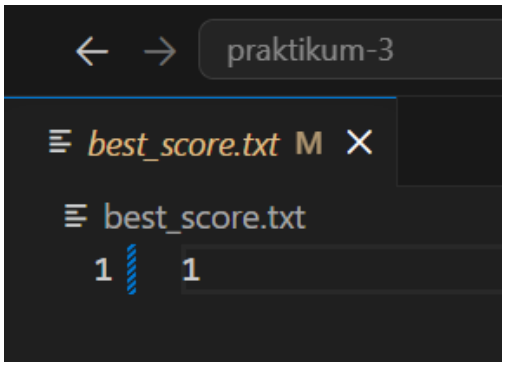
Gambar 2. 10 Program Fitur Waktu Game pada Program Bahasa Python

5. Penyimpanan Best Score

Fitur tambahan lainnya adalah penyimpanan skor terbaik ke dalam file. Skor terbaik ditentukan berdasarkan jumlah percobaan paling sedikit.

Pada bahasa C, digunakan file "best_score.txt" untuk menyimpan skor:

- Program membaca skor lama dari file
- Membandingkan dengan skor saat ini
- Menyimpan skor baru jika lebih baik



Gambar 2. 11 Fitur Penyimpanan Best Score pada Program Bahasa C

```c
FILE *fp = fopen("best_score.txt", "r");
int best = 0;

if (fp != NULL) {
    fscanf(fp, "%d", &best);
    fclose(fp);
}

if (best == 0 || attempts < best) {
    printf("Best score baru!\n");
    fp = fopen("best_score.txt", "w");
    fprintf(fp, "%d", attempts);
    fclose(fp);
} else {
    printf("Best score saat ini: %d\n", best);
}

return 0;
```

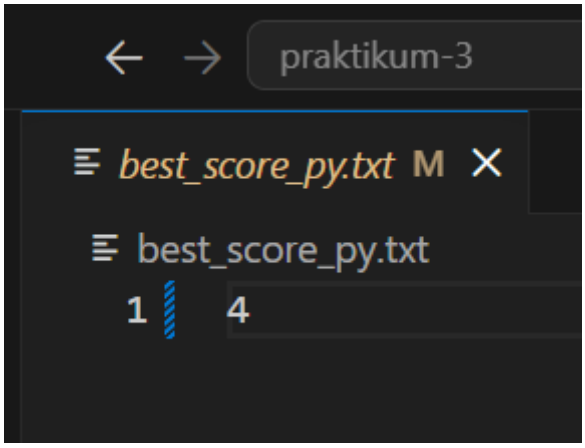
Gambar 2. 12 Program Best Score pada Program Bahasa C

Fitur best score pada program diimplementasikan dengan menggunakan file eksternal bernama "best_score.txt". Program terlebih dahulu membaca nilai skor terbaik yang tersimpan di dalam file menggunakan fungsi fopen() dan fscanf().

Nilai tersebut kemudian dibandingkan dengan jumlah percobaan yang dilakukan oleh pengguna pada permainan saat ini. Jika jumlah percobaan saat ini lebih kecil (lebih baik) atau belum terdapat data sebelumnya, maka program akan menyimpan skor baru ke dalam file menggunakan fungsi fprintf().

Sebaliknya, jika skor saat ini tidak lebih baik, maka program hanya akan menampilkan skor terbaik yang sudah ada. Dengan demikian, fitur ini memungkinkan penyimpanan data skor secara permanen dan dapat digunakan kembali pada permainan berikutnya.

Pada Python, konsep yang sama diterapkan menggunakan file "best_score_py.txt". Fitur ini memungkinkan program untuk menyimpan performa terbaik pengguna secara permanen.



Gambar 2. 13 Fitur Penyimpanan Best Score pada Program Bahasa Python

```
def load_best():
    try:
        with open("best_score_py.txt", "r") as f:
            return int(f.read())
    except:
        return None
```

Gambar 2. 14 Program Penyimpanan Best Score pada Program Bahasa Python

Gambar tersebut menunjukkan implementasi fitur penyimpanan best score pada program bahasa Python. Pada bagian atas terlihat file "best_score_py.txt" yang berfungsi sebagai media penyimpanan skor terbaik, di mana nilai yang tersimpan merupakan jumlah percobaan paling sedikit yang pernah dicapai oleh pengguna.

Pada bagian bawah ditampilkan potongan kode fungsi load_best(), yang digunakan untuk membaca nilai best score dari file. Program membuka file dalam mode baca, kemudian mengonversi isi file menjadi tipe integer. Jika file belum tersedia atau terjadi kesalahan, fungsi akan mengembalikan nilai None.

Fitur ini memungkinkan program untuk menyimpan dan mengambil data skor secara permanen, sehingga best score dapat digunakan kembali pada permainan berikutnya. Dengan demikian, permainan menjadi lebih interaktif karena pengguna dapat melihat dan berusaha mengalahkan skor terbaik yang telah dicapai sebelumnya.

2.2. Analisis Bug pada Program Bahasa C

Pada proses debugging, dilakukan pengujian terhadap program dengan sengaja membuat kesalahan pada rumus random, yaitu:

```
secret = rand() % max;
```

Gambar 2. 15 Proses Debugging Rumus Random pada Program Bahasa C

Pada Gambar 2.15. tersebut digunakan untuk menghasilkan bilangan acak. Fungsi rand() menghasilkan bilangan acak bertipe integer, kemudian operator modulus (%) digunakan untuk membatasi nilai agar berada dalam rentang tertentu. Namun, penggunaan rand() % max hanya menghasilkan nilai dalam rentang 0 hingga (max - 1).

Pada program ini, nilai yang digunakan adalah min = 50 dan max = 100, sehingga seharusnya bilangan acak yang dihasilkan berada dalam rentang 50 hingga 100. Akan tetapi, dengan rumus rand() % max, rentang yang dihasilkan justru menjadi 0 hingga 99.

Akibatnya, nilai yang dihasilkan dapat berada di bawah batas minimum yang telah ditentukan. Sebagai contoh, program dapat menghasilkan nilai seperti 12 atau 30, yang jelas berada di luar rentang yang seharusnya. Hal ini menyebabkan ketidaksesuaian antara logika program dengan output yang dihasilkan.

Bug ini terjadi karena rumus tersebut tidak memperhitungkan nilai minimum (min). Operator modulus hanya membatasi nilai maksimum, tetapi tidak menggeser nilai agar dimulai dari batas minimum yang diinginkan.

Berikut contoh hasil pengujian menunjukkan bahwa nilai secret yang dihasilkan dapat berada di luar rentang yang seharusnya, yaitu:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    powershell

PS C:\praktikum-3> gcc debug_guess.c.c -o guess
PS C:\praktikum-3> .\guess.exe
SECRET: 45
PS C:\praktikum-3>
```

Gambar 2. 16 Hasil Pengujian Debugging pada Program Bahasa C

Berdasarkan hasil pengujian program bahasa C, diperoleh output sebagai berikut:

SECRET: 45

Nilai tersebut dihasilkan dari penggunaan rumus: *secret = rand() % max;* seperti contoh yang ditunjukkan pada Gambar 2.15.

Pada program, nilai yang ditentukan adalah min = 50 dan max = 100, sehingga seharusnya nilai secret berada dalam rentang 50 hingga 100. Namun, hasil pengujian menunjukkan bahwa nilai yang dihasilkan adalah 45, yang berada di bawah batas minimum yang ditentukan.

Hal ini membuktikan bahwa terdapat bug pada perhitungan bilangan acak. Kesalahan terjadi karena penggunaan operator modulus (%) hanya membatasi nilai maksimum, tetapi tidak memperhitungkan nilai minimum. Akibatnya, rentang nilai yang dihasilkan menjadi 0 hingga (max - 1), yaitu 0 hingga 99.

Karena nilai minimum tidak diperhitungkan, maka nilai yang dihasilkan dapat berada di luar rentang yang diharapkan, seperti pada hasil pengujian ini yang menghasilkan nilai 45.

Untuk memastikan adanya bug, dilakukan teknik debugging dengan menampilkan nilai variabel secret menggunakan fungsi printf. Dari hasil tersebut dapat disimpulkan bahwa program belum menghasilkan nilai yang sesuai dengan ketentuan yang ditetapkan.

Dengan demikian, hasil pengujian ini membuktikan bahwa penggunaan rumus rand() % max tidak sesuai untuk menghasilkan bilangan acak dalam rentang tertentu yang memiliki batas minimum.

2.3. Perbaikan Bug pada Program Bahasa C

Perbaikan bug pada program bahasa C dilakukan, berikut contoh gambar:

```
secret = rand() % (max - min + 1) + min;
```

Gambar 2. 17 Perbaikan Bug (Fix) pada Program Bahasa C

Sebelumnya, program menggunakan rumus rand() % max yang hanya membatasi nilai maksimum tanpa memperhitungkan nilai minimum. Akibatnya, nilai yang dihasilkan berada dalam rentang 0 hingga (max - 1), sehingga tidak sesuai dengan batas minimum yang telah ditentukan dalam program.

Untuk mengatasi permasalahan tersebut, dilakukan perbaikan dengan menyesuaikan rumus pembangkitan bilangan acak agar sesuai dengan rentang yang diinginkan, yaitu dari min hingga max.

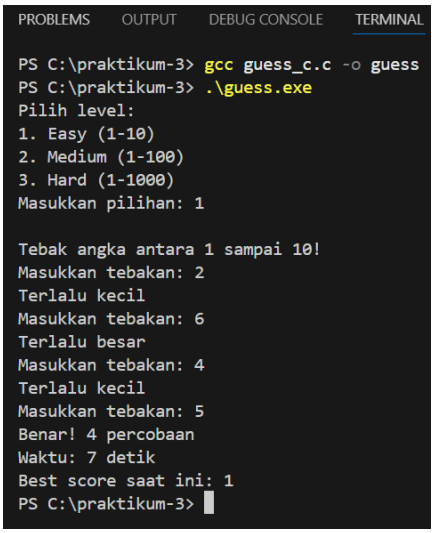
Rumus (max - min + 1) digunakan untuk menentukan panjang rentang bilangan yang diinginkan. Operator modulus (%) kemudian memastikan bahwa nilai acak yang dihasilkan berada dalam rentang tersebut. Selanjutnya, penambahan nilai min berfungsi untuk menggeser hasil agar dimulai dari nilai minimum yang telah ditentukan.

Dengan demikian, nilai yang dihasilkan oleh program akan selalu berada dalam rentang min hingga max secara inklusif. Sebagai contoh, jika min = 50 dan max = 100, maka nilai yang dihasilkan akan selalu berada dalam rentang 50 hingga 100.

Setelah dilakukan perbaikan, hasil pengujian menunjukkan bahwa tidak ada lagi nilai yang berada di luar batas minimum maupun maksimum. Hal ini membuktikan bahwa bug telah berhasil diperbaiki dan program telah berjalan sesuai dengan logika yang diharapkan.

Dengan perbaikan tersebut, konsistensi antara nilai yang dihasilkan dan rentang yang ditampilkan pada program dapat terjaga, sehingga program menjadi lebih akurat dan tidak membingungkan pengguna.

Dengan perbaikan tersebut, nilai yang dihasilkan selalu berada dalam rentang yang sesuai, berikut contoh gambar:



Gambar 2. 18 Hasil Pengujian Perbaikan Bug pada Program Bahasa C

Gambar 2.18 tersebut menunjukkan bahwa nilai secret berada dalam rentang yang sesuai, yaitu antara 1 hingga 10. Hal ini membuktikan bahwa perbaikan pada rumus random telah berhasil mengatasi bug yang sebelumnya menyebabkan nilai berada di luar rentang.

Logika permainan juga berjalan dengan baik dan konsisten. Ketika pengguna memasukkan angka yang lebih kecil dari nilai secret, program memberikan respon "terlalu kecil", sedangkan jika lebih besar akan memberikan respon "terlalu besar". Hal ini menunjukkan bahwa proses percabangan dalam program telah berjalan sesuai dengan yang diharapkan.

Selain itu, program mampu mencatat jumlah percobaan, yaitu sebanyak 4 kali, serta waktu yang dibutuhkan, yaitu 7 detik. Program juga menampilkan informasi best score, yang menunjukkan bahwa fitur penyimpanan skor berjalan dengan baik.

Dari hasil pengujian tersebut dapat disimpulkan bahwa setelah dilakukan perbaikan, program telah berfungsi secara optimal. Nilai yang dihasilkan sesuai dengan rentang yang ditentukan, serta alur permainan berjalan secara logis dan tidak menimbulkan kebingungan bagi pengguna.

2.4. Analisis Bug pada Program Bahasa Python

Pada proses debugging program Python, ditemukan kesalahan pada penggunaan fungsi pembangkit bilangan acak, yaitu:

```
secret = random.randint(0, max_v)
```

Gambar 2. 19 Proses Debugging Rumus Random pada Program Bahasa C

Pada Gambar 2.19 fungsi random.randint(a, b) digunakan untuk menghasilkan bilangan acak dalam rentang a hingga b (inklusif). Pada kode tersebut, nilai batas bawah yang digunakan adalah 0, sedangkan program mendefinisikan rentang permainan menggunakan variabel min_v dan max_v, misalnya min_v = 50 dan max_v = 100.

Kesalahan ini menyebabkan rentang nilai yang dihasilkan menjadi tidak sesuai dengan ketentuan permainan. Secara logika, program seharusnya menghasilkan angka dalam rentang 50 hingga 100. Namun, dengan penggunaan random.randint(0, max_v), angka yang dihasilkan berada dalam rentang 0 hingga 100. Akibatnya, nilai yang dihasilkan dapat berada di bawah batas minimum yang telah ditentukan.

Hal ini dibuktikan pada hasil pengujian, di mana nilai secret yang dihasilkan adalah 37, sedangkan rentang yang seharusnya adalah 50–100. Kondisi ini menunjukkan bahwa program memiliki bug karena menghasilkan nilai di luar batas yang diharapkan.Untuk memastikan letak kesalahan, dilakukan teknik debugging dengan menambahkan output untuk menampilkan nilai variabel secret secara langsung. Dengan cara ini, dapat diamati bahwa nilai yang dihasilkan tidak selalu sesuai dengan rentang yang ditentukan.

Berikut hasil pengujian debugging pemrograman Bahasa Python :

```
PROBLEMS    OUTPUT    TERMINAL    ···                    powershell

PS C:\praktikum-3> python debug_guess.py.py
SECRET (BUG): 37
Tebak angka antara 50 sampai 100!
Masukkan tebakan: 50
Terlalu besar.
Masukkan tebakan: 40
Terlalu besar.
Masukkan tebakan: 34
Terlalu kecil.
Masukkan tebakan: 37
Benar! 4 percobaan.
PS C:\praktikum-3>
```

Gambar 2. 20 Hasil Pengujian Debugging pada Program Bahasa Python

Berdasarkan hasil pengujian program Python, diperoleh pada gambar 2.20 yaitu,

SECRET (BUG): 37

Tebak angka antara 50 sampai 100!

Hasil tersebut menunjukkan bahwa nilai secret yang dihasilkan adalah 37, sedangkan program menetapkan bahwa rentang angka yang harus ditebak adalah antara 50 hingga 100. Hal ini menunjukkan adanya ketidaksesuaian antara nilai yang dihasilkan dengan batasan yang ditentukan dalam program. Kesalahan ini disebabkan oleh penggunaan fungsi random yang tidak tepat, seperti contoh pada gambar 2.19.

Fungsi tersebut menghasilkan bilangan acak dalam rentang 0 hingga max_v. Karena nilai minimum yang digunakan adalah 0, maka angka yang dihasilkan dapat berada di bawah nilai minimum yang seharusnya, yaitu min_v = 50. Oleh karena itu, nilai seperti 37 masih mungkin muncul, meskipun tidak sesuai dengan aturan permainan.

Akibat dari bug ini, logika permainan menjadi tidak konsisten. Program meminta pengguna menebak angka dalam rentang 50–100, namun angka yang sebenarnya dapat berada di luar rentang tersebut. Hal ini dapat membingungkan pengguna, karena petunjuk yang

diberikan program (“terlalu kecil” atau “terlalu besar”) tidak sesuai dengan ekspektasi berdasarkan rentang yang ditampilkan.

Sebagai contoh, ketika pengguna memasukkan angka 50, program memberikan respon “terlalu besar”, yang berarti angka sebenarnya lebih kecil dari 50. Hal ini bertentangan dengan informasi awal yang menyatakan bahwa angka berada dalam rentang 50–100.

2.5. Perbaikan Bug pada Program Bahasa Python

Untuk memperbaiki kesalahan tersebut, parameter fungsi random perlu disesuaikan dengan variabel yang digunakan dalam program, yaitu:

```
secret = random.randint(min_v, max_v)
```

Gambar 2. 21 Perbaikan Bug (Fix) pada Program Bahasa Python

Perubahan ini bertujuan agar batas bawah (min_v) dan batas atas (max_v) sesuai dengan rentang yang digunakan dalam permainan. Fungsi random.randint(a, b) akan menghasilkan bilangan acak dalam rentang a hingga b (inklusif), sehingga dengan menggunakan min_v dan max_v, nilai yang dihasilkan akan selalu berada dalam rentang yang benar.

Setelah dilakukan perbaikan, hasil pengujian menunjukkan bahwa nilai secret tidak lagi berada di luar batas minimum. Hal ini membuktikan bahwa bug telah berhasil diperbaiki dan logika program kembali sesuai dengan aturan permainan.

Dengan demikian, perbaikan ini memastikan bahwa konsistensi antara tampilan rentang pada program dengan nilai yang dihasilkan tetap terjaga, sehingga program menjadi lebih akurat dan tidak membingungkan pengguna.

Dengan perbaikan tersebut, nilai yang dihasilkan selalu berada dalam rentang yang sesuai, berikut contoh gambar:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\praktikum-3> python guess_py.py
Pilih level:
1. Easy (1-10)
2. Medium (1-100)
3. Hard (1-1000)
Masukkan pilihan: 1

Tebak angka antara 1 sampai 10!
Masukkan tebakan: 4
Terlalu kecil.
Masukkan tebakan: 5
Terlalu kecil.
Masukkan tebakan: 7
Terlalu besar.
Masukkan tebakan: 6
Benar! 4 percobaan.
Waktu: 6.42 detik
Best score baru!
PS C:\praktikum-3>
```

Gambar 2. 22 Hasil Pengujian Perbaikan Bug pada Program Python

Hasil Gambar 2.22 menunjukkan bahwa nilai secret berada dalam rentang yang sesuai, yaitu antara 1 hingga 10. Hal ini membuktikan bahwa perbaikan yang dilakukan telah berhasil mengatasi bug pada program.

Sebelumnya, program menghasilkan nilai di luar rentang yang ditentukan karena penggunaan parameter fungsi random yang tidak tepat. Setelah diperbaiki menjadi:
secret = random.randint(min_v, max_v) seperti pada contoh gambar 2.21.
nilai yang dihasilkan selalu berada dalam batas minimum dan maksimum yang telah ditentukan.

Selain itu, logika permainan juga berjalan dengan baik dan konsisten. Ketika pengguna memasukkan angka yang lebih kecil dari nilai secret, program memberikan respon "terlalu kecil", dan ketika lebih besar, program memberikan respon "terlalu besar". Hal ini menunjukkan bahwa alur percabangan dalam program berjalan sesuai dengan yang diharapkan.

Pada percobaan tersebut, pengguna berhasil menebak angka dalam 4 percobaan dengan waktu 6.42 detik. Program juga berhasil menyimpan best score baru, yang menunjukkan bahwa fitur penyimpanan skor berfungsi dengan baik.

Dengan demikian, dapat disimpulkan bahwa setelah dilakukan perbaikan, program telah berjalan secara optimal, menghasilkan nilai yang sesuai dengan rentang yang ditentukan, serta memberikan pengalaman pengguna yang konsisten dan tidak membingungkan.

2.6. Teori Perbedaan For pada Program Bahasa C dan Python

1. Konsep Dasar

Bahasa C

1) Perulangan for bersifat berbasis kondisi
2) Menentukan awal, kondisi, dan perubahan (increment/decrement)

Bahasa Python

1) Perulangan for bersifat berbasis iterasi (sequence)
2) digunakan untuk mengulang data seperti list, string, atau range

2. Sintaks

Bahasa C: *for (inisialisasi; kondisi; increment)*

Bahasa Python: *for variabel in sequence*

3. Contoh Program

Bahasa C

```
#include <stdio.h>

int main() {
    for (int i = 0; i < 5; i++) {
        printf("%d\n", i);
    }
    return 0;
}
```

Penjelasan

i = 0 : nilai awal

i < 5 : kondisi

i++ : penambahan

Output

0

1

2

3

4

Bahasa Python

```
for i in range(5):
    print(i)
```

Penjelasan

range(5) menghasilkan angka 0–4

i otomatis mengambil nilai dari range

Output

0

1

2

3

4

Perbedaan Utama

Aspek	C	Python
Cara Kerja	Berbasis Kondisi	Berbasis Iterasi
Sintaks	Lebih Kompleks	Lebih Sederhana
Penggunaan	Manual (awal,kondisi,increment)	Otomatis (pakai range/sequence)
Fleksibilitas	Lebih Fleksibel)	Lebih Mudah Dibaca

Kesimpulan

Perulangan for pada bahasa C memberikan kontrol yang lebih detail karena menggunakan tiga komponen utama (inisialisasi, kondisi, dan increment). Sedangkan pada Python, perulangan for lebih sederhana karena langsung melakukan iterasi terhadap suatu sequence, sehingga lebih mudah dipahami dan ditulis.

3. KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa program game tebak angka berhasil diimplementasikan dengan baik menggunakan bahasa pemrograman C dan Python. Program ini mencakup berbagai konsep dasar pemrograman seperti percabangan, perulangan, penggunaan fungsi random, serta pengolahan input dan output.

Pengembangan program dengan menambahkan fitur level permainan (easy, medium, hard) membuat game menjadi lebih variatif dan menantang. Selain itu, fitur pengukuran waktu memungkinkan pengguna mengetahui durasi dalam menyelesaikan permainan, sedangkan fitur best score memungkinkan penyimpanan skor terbaik secara permanen menggunakan file.

Proses debugging yang dilakukan pada kedua bahasa menunjukkan pentingnya ketelitian dalam penulisan logika program, khususnya pada pembangkitan bilangan acak. Kesalahan pada penggunaan rumus random dapat menyebabkan hasil yang tidak sesuai dengan rentang yang ditentukan. Setelah dilakukan perbaikan, program mampu menghasilkan nilai yang sesuai dan berjalan secara optimal.

Selain itu, perbandingan perulangan for pada bahasa C dan Python menunjukkan bahwa kedua bahasa memiliki perbedaan dalam sintaks dan cara kerja, di mana bahasa C lebih fleksibel namun kompleks, sedangkan Python lebih sederhana dan mudah dipahami.

Dengan demikian, praktikum ini memberikan pemahaman yang lebih mendalam mengenai dasar-dasar pemrograman, proses debugging, serta perbedaan implementasi antara bahasa C dan Python dalam menyelesaikan suatu permasalahan.